



BÁO CÁO BÀI THỰC HÀNH SỐ 1

Data Lab: Tính toán Bits

Môn học: NT209 - Lập trình Hệ thống

Sinh viên thực hiện	Lê Ngọc Phương Thảo - 23521467 Bùi Nguyễn Minh Anh - 24520086
Thời gian thực hiện	12/03/2026 – 18/03/2026
Số câu đã hoàn thành	9/9

NỘI DUNG BÁO CÁO

1. Tính toán bit đơn giản

Các hàm này có đầu vào là các số nguyên 32 bit và đầu ra cũng là các số nguyên 32 bit được tính toán bằng các toán tử trên bit.

- Số câu đố: **5**
- Điểm tối đa: **10**

Viết các hàm với các yêu cầu như sau:

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
1.1	int negative (x)	Tính giá trị của $-x$ không dùng dấu $-$ • Input: Số nguyên x • PT: negative(0) == 0 && negative(9) == -9 && negative(-5) == 5	1	8

Trình bày: Trong kiến trúc máy tính, số nguyên có dấu được biểu diễn dưới dạng số bù 2.

Giải thích: Để biểu diễn dưới dạng số bù 2, cần sử dụng toán tử NOT và cộng thêm 1 vào kết quả.

Hình chụp minh chứng:

```
3 > // 1.1 negative(x): tra ve -x...
5 int negative(int x)
6 {
7     return (~x) + 1;
8 }
```

1.1 negative Pass.

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
1.2	int cal50x (x)	Tính giá trị $50 \cdot x$ mà không được dùng phép nhân. • Input: Số nguyên x (giới hạn sao cho $50x$ biểu diễn đủ trong số có dấu 4 byte) • PT: cal50x(0) == 0 && cal50x(10) == 500 && cal50x(-5) == -250	1	10

Trình bày: Trong kiến trúc máy tính, phép nhân với hằng số có thể thay bằng phép dịch bit với $x \ll n = x \times 2^n$.

Giải thích: Ta có $50 = 32 + 16 + 2 = 2^5 + 2^4 + 2^1$ nên phép nhân 50 với x sẽ tương đương với $(x \ll 5) + (x \ll 4) + (x \ll 1)$.

Hình chụp minh chứng:

```
10 > // 1.2 cal50x(x): tra ve x * 50...
13 v int cal50x(int x){
14     return (x << 5) + (x << 4) + (x << 1);
15 }
```

1.2 cal50x Pass.

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
1.3	int getByte(x,n)	Trả về byte thứ n của x (các byte được đánh thứ tự từ 0 đến 3 từ phải sang trái). Input: - Số x nguyên (giới hạn 4 byte) - Số n ($0 \leq n \leq 3$) - PT: <code>getByte(26,0) == 0x1A</code> && <code>getByte(0x11223344,1) == 0x33</code>	2	10

Trình bày: Để lấy một byte cụ thể, cần đưa nó về vị trí thấp nhất rồi dùng mặt nạ lọc lấy 8 bit đó (1 byte gồm 8 bit).

Giải thích: Trích xuất byte thứ n của số nguyên x, sử dụng phép dịch phải để đưa byte mục tiêu về vị trí ngoài cùng bên phải. Thực hiện phép `& 0xFF` để giữ lại đúng 8 bit của byte đó và xóa bỏ tất cả các bit phía trước.

Hình chụp minh chứng:

```
17 > // 1.3 getByte(x, n): lay byte thu n (0-indexed, tu phai sang trai)
    cua x...
20 int getByte(int x, int n)
21 {
22     return (x >> (n << 3)) & 0xFF;
23 }
```

1.3 getByte Pass.

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
1.4	int flipByte (x,n)	Lật 1 byte thứ n của số nguyên x (các byte được đánh theo thứ tự từ 0 đến 3 từ phải sang trái), từ bit 0 thành 1 và ngược lại. - Input: số nguyên x, số n ($0 \leq n \leq 3$) - PT: flipByte(10,0)==245 && flipByte(0,1)==65280 && flipByte(0x5501,1)==0xaa01	3	20

Trình bày: Để lật được một byte thứ n của số nguyên, ta sử dụng phép XOR (^) với 1 sẽ giúp đảo bit từ 0 thành 1 và ngược lại, còn XOR với 0 sẽ giữ nguyên.

Giải thích: Để lật byte thứ n của số nguyên x, ta tạo mask có 8 bit 1 tại vị trí đó: $0xFF \ll (n \ll 3)$ (vì mỗi byte = 8 bit). Khi $x \wedge \text{mask}$, các bit ở byte n bị đảo, với các bit khác giữ nguyên.

Hình chụp minh chứng:

```

25 > // 1.4 flipByte(x, n): dao nguoc (flip) tat ca 8 bit cua byte thu n
    trong x...
28 int flipByte(int x, int n) {
29     return x ^ (0xFF << (n << 3));
30 }

```

1.4 flipByte Pass.

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
1.5	int divpw2 (x,n)	Tính kết quả $x/2^n$ - Input: - Số x nguyên dương - Số n âm ($-31 \leq n \leq -1$) - PT: divpw2(10, -1) == 20 && divpw2(15, -2) == 60 && divpw2(2, -4) == 32 <u>Nâng cao:</u> Tính kết quả $x/2^n$ với mọi giá trị n ($-31 \leq n \leq 31$)	3/4	8/20

Trình bày: Trong máy tính, chia cho 2^n sẽ tương đương dịch phải n lần, nhân với 2^k sẽ tương đương dịch trái k lần. Với số có dấu thì khi chia cần thêm bias để làm tròn đúng.

Giải thích:

- Nếu $n \geq 0$: $x/2^n = (x + \text{bias}) \gg n$, với **bias** = $(x \gg 31) \& ((1 \ll n) - 1)$ để xử lý số âm.
- Nếu $n < 0$: $x/2^n = x * 2^{-n} = x \ll (-n)$. Ta dùng **mask** = $n \gg 31$ để phân biệt hai trường hợp ($n < 0$ thì **mask** = -1, ngược lại = 0), số đối của n là **negN** = $\sim n + 1 = -n$.
- Cuối cùng chọn kết quả bằng: $(\text{mulRes} \& \text{mask}) \mid (\text{divRes} \& \sim \text{mask})$.

Hình chụp minh chứng:

```
32 > // 1.5 divpw2(x, n): tính x / 2^n (n >= 0: chia, n < 0: nhân) ...
74 < int divpw2(int x, int n) {
75     int mask = n >> 31;
76     int negN = ~n + 1;
77     int bias = (x >> 31) & ((1 << n) + ~0);
78
79     int divRes = (x + bias) >> n;
80     int mulRes = x << negN;
81
82     return (mulRes & mask) | (divRes & ~mask);
83 }
```

1.5 divpw2 Advanced Pass.

2. Các phép kiểm tra dựa trên tính toán bit

Các hàm này có đầu vào là các số nguyên 32 bit và đầu ra là các giá trị đại diện cho True/False, tức là 0 hoặc 1 (1 bit), của phép đánh giá nào đó.

- Số câu đố: 4
- Điểm tối đa: 10

Viết các hàm với các yêu cầu như sau:

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
2.1	int isOpposite (x,y)	Kiểm tra 2 số x và y có đối nhau hay không (cùng độ lớn nhưng khác dấu). Trả về 1 nếu đối nhau, 0 nếu ngược lại. Lưu ý trường hợp số 0. • Input: Số nguyên x, y • PT: isOpposite(4,4)==0 && isOpposite(2,2)==1 && isOpposite(5,-6)==0 && isOpposite(0,0) == 0	2	8

Trình bày: Trong biểu diễn số bù 2, hai số đối nhau thỏa $y = -x = \sim x + 1$. Phép XOR cho kết quả 0 nếu hai số bằng nhau, và toán tử ! dùng để kiểm tra bằng 0.

Giải thích: $\sim y + 1$ là $-y$, nên $x \wedge (\sim y + 1) == 0 \Leftrightarrow x == -y$ (hai số đối nhau). $!(x \wedge (\sim y + 1))$ trả về 1 nếu điều này đúng. Tuy nhiên cần loại trường hợp (0,0) nên thêm $!!(x | y)$ để đảm bảo ít nhất một số khác 0.

Hình chụp minh chứng:

```
85 > // 2.1 isOpposite(x, y): kiểm tra x và y là 2 số đối nhau (x = -y) ...
89 int isOpposite(int x, int y) {
90     return !(x ^ (~y + 1)) & !((x | y));
91 }
```

2.1 isOpposite Pass.

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
2.2	int is16x (x)	Kiểm tra 1 số nguyên x có chia hết cho 16 hay không. Trả về 1 nếu chia hết, không thì trả về 0. • Input: Số nguyên x không âm • PT: is16x(16)==1 && is16x(3)==0 && is16x(48)==1	2	10

Trình bày: Trong hệ nhị phân, các số chia hết cho 2^n sẽ có ít nhất n chữ số 0 ở cuối cùng. Vì 16 là 2^4 nên một số chia hết cho 16 chắc chắn có 4 bit cuối là 0000.

Giải thích: Dùng mask 0xF để lấy 4 bit cuối của x, dùng & 0xF trả về giá trị của 4 bit cuối. Nếu x chia hết cho 16, kết quả này bằng 0. Vì đề yêu cầu trả về 1 nếu chia hết, t dùng toán tử ! để chuyển 0 thành 1 và các giá trị khác 0 thành 0.

Hình chụp minh chứng:

```
93 > // 2.2 is16x(x): kiểm tra x là bội số của 16...
96 int is16x(int x) {
97     return !(x & 0xF);
98 }
```

2.2 is16x Pass.

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
2.3	int isPositive(x)	Trả về 1 nếu x dương ($x > 0$) - Input: Số nguyên x - PT: <code>isPositive(10)==1 && isPositive(-5)==0 && isPositive(0)==0</code>	3	15

Trình bày: Trong số bù 2, bit dấu là bit cao nhất: 0 nếu số không âm, 1 nếu số âm. Dịch phải số học `x >> 31` sẽ đưa bit dấu về toàn bộ.

Giải thích: `!!x` đảm bảo $x \neq 0$ (loại trường hợp 0), còn `!(x >> 31)` trả về 1 khi x không âm (bit dấu = 0).

Hình chụp minh chứng:

```
100 > // 2.3 isPositive(x): kiểm tra x > 0, trả về 0 nếu x <= 0...
104 v int isPositive(int x) {
105     return !!x & !(x >> 31);
106 }
```

2.3 isPositive Pass.

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
2.4	int isGE2n(x,n)	Trả về 1 nếu $x \geq 2^n$ Input:	3	20

		- Số x nguyên dương - Số n ($0 \leq n \leq 30$) - PT: <code>isGE2n(15,1)==1 && isGE2n(8,3)==1 && isGE2n(12,4)==0</code>		
--	--	--	--	--

Trình bày: Giá trị 2^n trong hệ nhị phân là một số có bit thứ n bằng 1 và các bit sau nó bằng 0. Suy ra, một số $x > 2^n$ nghĩa là x phải có ít nhất một bit bằng 1 từ vị trí n trở về bên trái.

Giải thích: Thực hiện phép dịch phải `x >> n` để loại bỏ bit đầu tiên bên phải. Nếu sau khi dịch mà kết quả vẫn còn giá trị (khác 0), nghĩa là ban đầu x có giá trị tại vị trí n hoặc cao hơn, tức là x lớn hơn hoặc bằng 2^n . Tiếp tục sử dụng toán tử `!!` để biến mọi giá trị khác 0 thành 1 và giữ nguyên 0 thành 0.

Chụp hình minh chứng:

```

108 > //2.4 isGE2n(x, n): kiểm tra x >= 2^n...
115 int isGE2n(int x, int n) {
116     return !(x >> n);
117 }

```

2.4 isGE2n Pass.